Intelligent Multilingual Spell Checking Suggestion System

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by

B. VR VR SrisaiHarini (2520030384)

K. Sai Samyuktha (2520030443)

B. Devi Charitha (2520030462)

Team Number: 10

Section: 4

Under the guidance of

Mrs. J. Yamini Devi



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY/ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

The Mini Project Report entitled "**Intelligent Multilingual Spell Checking Suggestion System"** is a record of Bonafide work of **Name –B. VR VR SrisaiHarini (2520030384), Name – K. Sai Samyuktha (2520030443), Name –B. Devi Charitha (2520030462)**submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering/Computer Science and Information Technology/ Artifical Intelligence & data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Name –B. VR VR SrisaiHarini

Roll.No.2520030384,

Name – K.Sai Samyuktha

Roll.No.2520030443,

Name –B. Devi Charitha

Roll.No.2520030462

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**Intelligent Spell Checker Using Levenstein Edit Distance and Data Structures"** is a bonafide work done and submitted by **Name –B. VR VR SrisaiHarini Roll.No.2520030384, Name – K.Sai Samyuktha Roll.No.2520030443, Name –B. Devi Charitha Roll.No.2520030462** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering/ Computer Science and Information Technology/Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, <<Mrs. J. Yamini Devi >>, <CSE >> which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Rao (CSE)/Dr. Khaja Shareef (CSIT)/ Dr. Sreenath Itikela (AI&DS)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Name –B. VR VR SrisaiHarini

Roll.No.2520030384,

Name – K.Sai Samyuktha

Roll.No.2520030443,

Name –B. Devi Charitha

Roll.No.2520030462

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	7-8
2.	Introduction	9-10
3.	Literature survey	11-13
4.	Problem Statement and Objectives	14-15
5.	Hardware and Software requirements	16
6.	Implementation	17-18
7.	Results & Discussion	19-21
8.	Conclusion & Future Scope	22-24
9.	References	25-26

1.ABSTRACT

The Intelligent Spell Checker is a console-based document processing system developed using Data Structures and Algorithms to automatically identify and correct spelling mistakes in a text document. Unlike a basic spell checker that accepts a single word and returns a nearby dictionary word, this project accepts the path of an entire document, analyzes its contents word by word, identifies incorrect spellings, determines suitable corrections, and generates a new corrected document without modifying the original file.

The main algorithm used in the project is Levenshtein Edit Distance, implemented using Dynamic Programming. The algorithm calculates the minimum number of character-level operations required to transform an incorrect word into a valid dictionary word. The supported operations are insertion, deletion, and substitution. The project also uses a Trie for efficient dictionary word lookup, a HashSet for dictionary storage, and a HashMap for grouping dictionary words according to their lengths.

When an incorrect word is detected, the system compares it with suitable dictionary candidates and selects a correction with the minimum edit distance within a defined threshold. The system also performs backtracking on the Dynamic Programming table to display the edit operations involved in the correction.

The final corrected document is automatically created as a separate output file. The system also displays statistics such as total words, incorrect words, corrections made, suggested corrections, and edit distances.

Team Name: Intelligent Spell Checker Using Levenshtein Edit Distance and Data Structures

Team Members:

Name –B. VR VR SrisaiHarini

Roll.No.2520030384,

Name – K.Sai Samyuktha

Roll.No.2520030443,

Name –B. Devi Charitha

Roll.No.2520030462

2. INTRODUCTION

Spelling mistakes are common when creating digital documents, assignments, reports, notes, and other forms of written content. Manually identifying and correcting every spelling mistake in a large document can be time-consuming and inefficient. A spell-checking system can automate this process by comparing words in a document with a dictionary of valid words.

Traditional simple spell-checking programs often work by accepting one word from the user and finding a similar word from a dictionary. However, such a system does not provide a complete document-level solution. The proposed **Intelligent Spell Checker** addresses this limitation by allowing the user to provide the path of a complete text document. The program reads the document, analyzes every word, detects possible spelling errors, calculates suitable corrections, and generates a corrected copy of the document.

The core algorithm of the project is **Levenstein Edit Distance**. It measures the minimum number of insertions, deletions, and substitutions required to transform one string into another. Dynamic Programming is used to calculate the edit distance efficiently by storing solutions to smaller subproblems.

For example, if the input contains the misspelled word:

libary

and the dictionary contains:

library

the algorithm identifies that inserting the character r transforms libary into library, resulting in an edit distance of 1.

The project combines multiple DSA concepts including **Trie, HashSet, HashMap, Dynamic Programming, String Processing, Searching, and Backtracking**. The system therefore demonstrates how fundamental DSA concepts can be combined to solve a practical real-world problem.

The project accepts a text document and dictionary as input, analyzes the document, corrects suitable spelling mistakes, and produces a new corrected document while preserving the original document.

3. LITERATURE SURVEY

3.1 Levenshtein Edit Distance

Vladimir Levenshtein introduced the mathematical foundation for the edit-distance metric in his work on correcting deletions, insertions, and substitutions in sequences. The metric provides a way to measure the minimum number of character-level operations needed to transform one sequence into another. This concept forms the core of the proposed Intelligent Spell Checker.

Relevance to our project:
The proposed system uses Levenshtein Edit Distance to determine how closely a misspelled word resembles a dictionary word. Dynamic Programming is used to calculate the minimum edit distance.

3.2 Peter Norvig's Spelling Corrector

Peter Norvig presented a compact spelling-correction approach that generates possible spelling variations and selects likely corrections using a dictionary and word probabilities. His work demonstrates how deletion, insertion, replacement, and other character-level changes can be used to generate spelling candidates.

Relevance to our project:
The project follows the general idea of comparing incorrect words with valid dictionary words, but instead of implementing a probability-based correction system, our project focuses on Levenshtein Edit Distance and DSA concepts.

3.3 Hunspell

Hunspell is a widely used open-source spell-checking and morphological-analysis system. It is used by applications including LibreOffice and various browsers. It supports dictionary-based spell checking and language-specific morphological processing.

Relevance to our project:
Hunspell demonstrates how dictionary-based spelling correction can be incorporated into practical software. Our project implements a smaller educational version focused specifically on DSA concepts rather than advanced linguistic morphology.

3.4 SymSpell

SymSpell is a spelling-correction and fuzzy-search algorithm designed to reduce the computational cost of generating and searching spelling candidates. It uses precomputed deletion-based candidates to achieve fast lookup for approximate matches.

Relevance to our project:
SymSpell demonstrates the importance of efficient candidate generation in spelling correction. Our project uses a simpler length-based candidate filtering approach together with a Trie and Levenshtein Dynamic Programming.

3.5 BK-Tree Based Approximate Searching

Burkhard and Keller introduced a tree-based approach for best-match searching in metric spaces. BK-Trees can be used for approximate string matching where edit distance is used as the metric.

Relevance to our project:

A BK-Tree could be used as an alternative data structure for finding dictionary words close to a misspelled word. Our current implementation instead uses a Trie and length-based candidate grouping to keep the project simpler and demonstrate multiple DSA concepts.

3.6 LanguageTool

LanguageTool is a modern writing-assistance system that provides spelling, grammar, style, and writing suggestions. It is available as a web editor and browser extensions and can analyze longer pieces of text.

Relevance to our project:

LanguageTool demonstrates the practical importance of document-level language checking. Our project focuses specifically on spelling correction using classical DSA techniques rather than grammar and style analysis.

Literature Survey Summary

Existing spell-checking systems range from simple dictionary-based correctors to advanced systems that use statistical models, morphological analysis, and optimized approximate-search algorithms. The proposed project focuses on the **algorithmic foundation of spelling correction**, particularly Levenshtein Edit Distance, Dynamic Programming, Trie-based lookup, and document processing.

4. Problem Statement and Objectives

4.1 Problem Statement

To develop a console-based intelligent spell checker that reads a complete text document, detects misspelled words using a dictionary, and automatically generates a corrected document using Levenshtein Edit Distance.

4.2 Problem Description

Basic spell-checking programs often process only individual words and return a possible dictionary match. They do not provide a complete document-level correction workflow. The proposed system analyzes an entire document, identifies incorrect words, calculates the minimum character edits required for suitable dictionary words, and creates a corrected copy of the document.

4.3 Objectives

The main objectives of the project are:

1. To develop a **console-based document spell-checking system**.
2. To allow users to provide the **path of a text document**.
3. To read and analyze the document word by word.
4. To maintain a dictionary of valid English words.
5. To efficiently determine whether a word exists in the dictionary.
6. To identify incorrect or misspelled words.

7. To calculate the similarity between incorrect words and dictionary words using **Levenshtein Edit Distance**.
8. To implement Levenshtein distance using **Dynamic Programming**.
9. To support the three major edit operations:
    - Insertion
    - Deletion
    - Substitution
10. To identify the individual edit operations using **backtracking**.
11. To select a suitable dictionary word based on minimum edit distance.
12. To avoid inappropriate corrections using an **edit-distance threshold**.
13. To preserve punctuation, spaces, and capitalization while correcting words.
14. To create a separate corrected document instead of modifying the original file.
15. To display useful statistics such as total words, incorrect words, and corrections made.
16. To demonstrate the practical application of multiple **Data Structures and Algorithms**.

5. HARDWARE AND SOFTWARE REQUIREMENTS

The report template specifically asks for hardware and software requirements.

5.1 Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Minimum 1 GHz processor
- At least 500 MB free storage
- Keyboard and monitor
- Internet connection is optional for development/reference purposes

5.2 Software Requirements

- Operating System: Windows 10/11 or equivalent
- Programming Language: **Java**
- Java Development Kit: **JDK 8 or above**
- IDE: **Visual Studio Code**
- Java Compiler: javac
- Java Runtime: java
- Input: .txt document
- Dictionary: .txt file containing valid words
- Output: Corrected .txt document

6. IMPLEMENTATION

The Intelligent Spell Checker is developed using Java and Data Structures and Algorithms to identify and correct spelling mistakes in text documents. It reads an input document and a dictionary file, detects misspelled words, finds suitable corrections using Levenshtein Edit Distance, and generates a separate corrected document.

6.1 File Handling and Dictionary Loading

The system accepts the paths of the input text file and dictionary file from the user. It loads the dictionary into memory and prepares the words for comparison. Java file-handling classes are used to read the files and save the corrected output.

6.2 Error Detection

The program reads the document and checks each word against the dictionary. Words that are not found are identified as possible spelling errors. Punctuation and spacing are preserved as far as supported by the implementation.

6.3 Spelling Correction Using Levenshtein Distance

The system uses the Levenshtein Edit Distance algorithm to find suitable corrections. It calculates the minimum number of character insertions, deletions, and substitutions required to transform a misspelled word into a dictionary word. Dynamic Programming is used to store intermediate results in a two-dimensional matrix, reducing repeated calculations.

6.4 Candidate Selection and Backtracking

The program compares dictionary candidates and selects a suitable word based on the minimum edit distance. Backtracking through the DP matrix can identify the character operations required for the correction. A distance threshold helps avoid replacing a word with an unrelated candidate.

6.5 Corrected Document Generation

After processing the document, the system replaces selected misspelled words and saves the result in a separate file named corrected_input.txt. The original document remains unchanged, allowing users to compare both versions.

6.6 Result Analysis

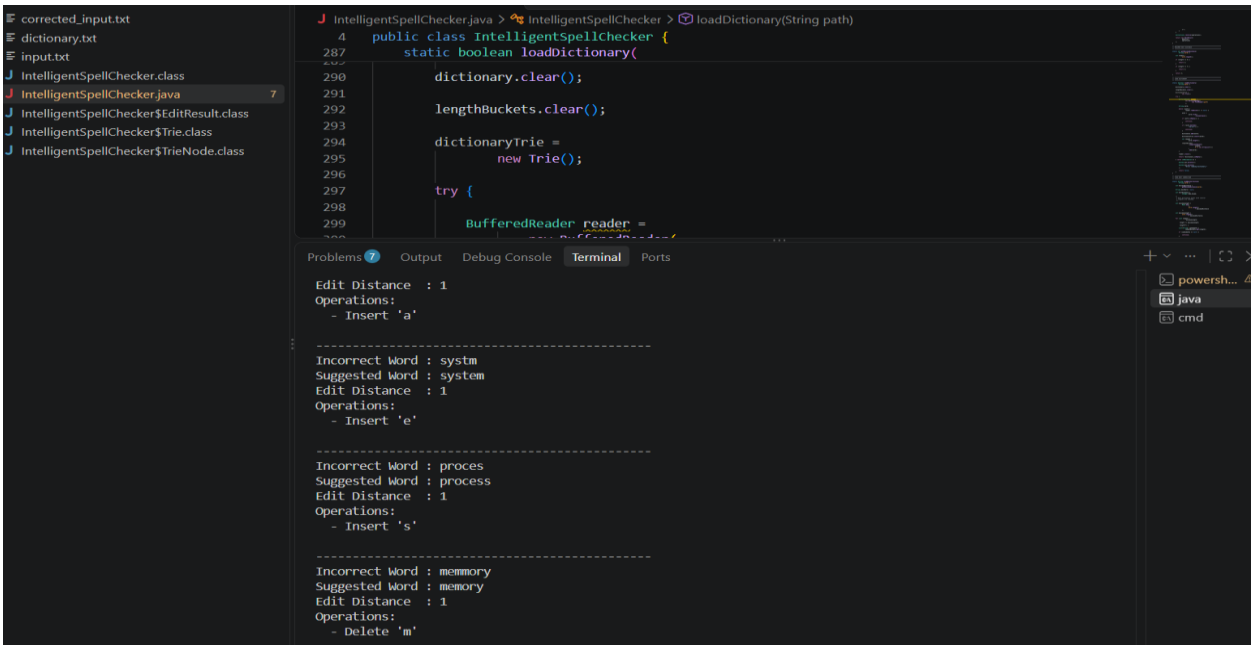
Finally, the program displays the total number of words processed, the number of possible spelling errors, the corrections made, and the output file location. This provides a simple summary of the spell-checking process.

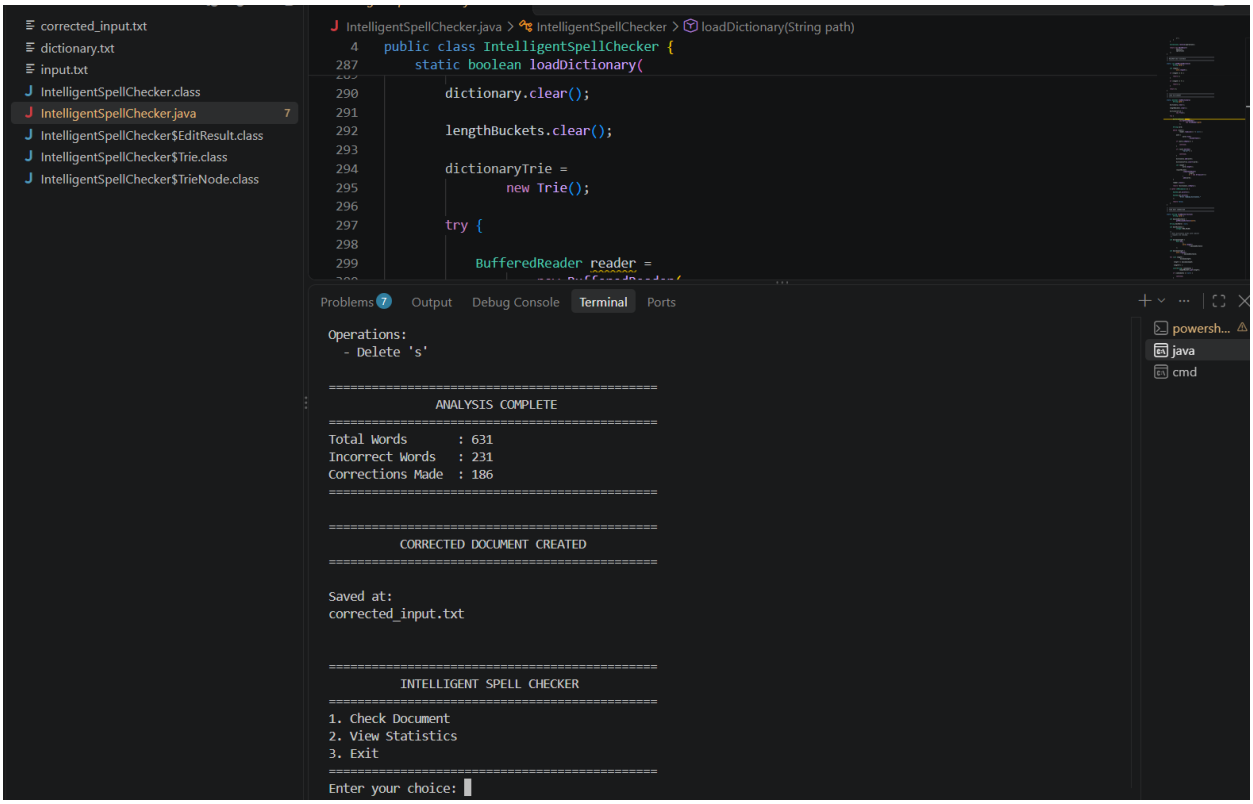
6.7 Data Structures and Algorithms Used

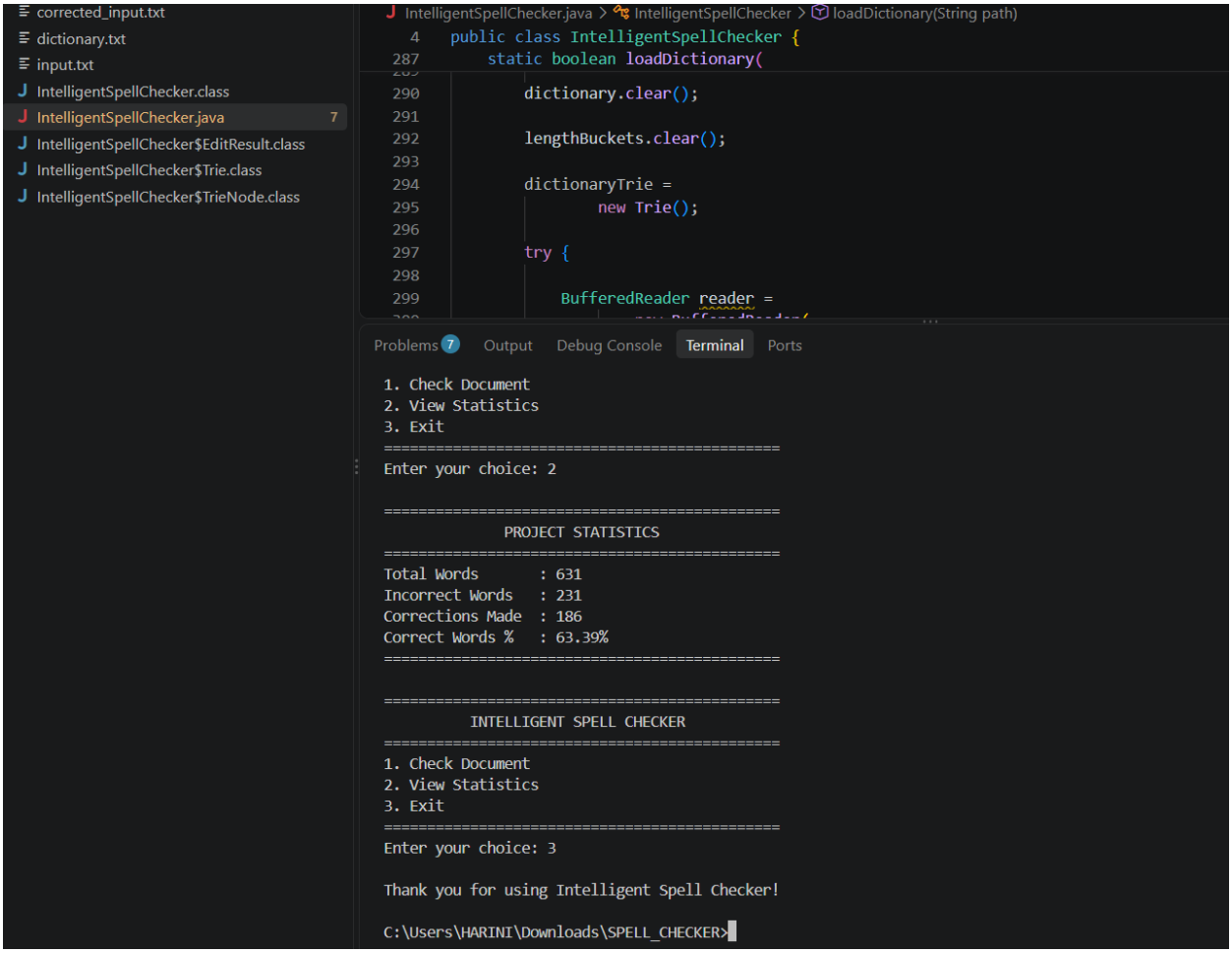
The project uses dictionary storage and lookup structures such as HashSet, and other structures such as Trie or HashMap only if included in the implementation. Dynamic Programming calculates edit distance, while backtracking identifies correction operations. File handling enables document reading and corrected output generation.

7. RESULTS & DISCUSSION

Output application run screen shots with their explanation in 2 lines.




corrected_input.txt
dictionary.txt
input.txt
IntelligentSpellChecker.class
IntelligentSpellChecker.java 7
IntelligentSpellChecker$EditResult.class
IntelligentSpellChecker$Trie.class
IntelligentSpellChecker$TrieNode.class
IntelligentSpellChecker.java > IntelligentSpellChecker > loadDictionary(String path)
4 public class IntelligentSpellChecker {
287 static boolean loadDictionary(
290 dictionary.clear();
292 lengthBuckets.clear();
294 dictionaryTrie =
295 new Trie();
297 try {
299 BufferedReader reader =
Problems 7 Output Debug Console Terminal Ports
powersh...
java
cmd
Operations:
- Delete 's'
ANALYSIS COMPLETE
Total Words : 631
Incorrect Words : 231
Corrections Made : 186
CORRECTED DOCUMENT CREATED
Saved at:
corrected_input.txt
INTELLIGENT SPELL CHECKER
1. Check Document
2. View Statistics
3. Exit
Enter your choice:


corrected_input.txt
dictionary.txt
input.txt
IntelligentSpellChecker.class
IntelligentSpellChecker.java 7
IntelligentSpellChecker$EditResult.class
IntelligentSpellChecker$Trie.class
IntelligentSpellChecker$TrieNode.class
IntelligentSpellChecker.java > IntelligentSpellChecker > loadDictionary(String path)
4 public class IntelligentSpellChecker {
287 static boolean loadDictionary(
290 dictionary.clear();
291
292 lengthBuckets.clear();
293
294 dictionaryTrie =
295 new Trie();
296
297 try {
298
299 BufferedReader reader =
Problems 7 Output Debug Console Terminal Ports
1. Check Document
2. View Statistics
3. Exit
==========
Enter your choice: 2
==========
PROJECT STATISTICS
==========
Total Words : 631
Incorrect Words : 231
Corrections Made : 186
Correct Words % : 63.39%
==========
==========
INTELLIGENT SPELL CHECKER
==========
1. Check Document
2. View Statistics
3. Exit
==========
Enter your choice: 3
Thank you for using Intelligent Spell Checker!
C:\Users\HARINI\Downloads\SPELL_CHECKER>

8. CONCLUSION & FUTURE SCOPE

8.1 Conclusion

The **Intelligent Spell Checker** successfully demonstrates the practical application of Data Structures and Algorithms to document processing and spelling correction. The system goes beyond simple single-word correction by accepting an entire text document, analyzing its contents, detecting misspelled words, identifying suitable dictionary words, and generating a corrected document.

The primary algorithm used in the project is **Levenshtein Edit Distance**, implemented using Dynamic Programming. The algorithm calculates the minimum number of insertions, deletions, and substitutions required to transform an incorrect word into a valid dictionary word. Backtracking is additionally used to identify the specific edit operations.

The project also demonstrates the use of a **Trie** for dictionary lookup, **HashSet** for dictionary storage, **HashMap** for candidate grouping, arrays for Dynamic Programming, and ArrayList for storing operations. These data structures work together to create a complete DSA-based application.

The system preserves the original document and creates a separate corrected file. It also provides useful statistics about the number of words analyzed and corrections performed.

Overall, the project demonstrates that fundamental DSA concepts can be combined to develop a practical and useful text-processing application.

8.2 FUTURE SCOPE

The project can be further enhanced in several ways:

1. Larger Dictionary

A much larger English dictionary can be integrated to improve correction coverage.

2. Word Frequency

Word-frequency information can be added so that when multiple words have the same edit distance, the most commonly used word can be selected.

3. BK-Tree

A BK-Tree can be introduced for efficient approximate dictionary searching based on edit distance.

4. SymSpell

The candidate-generation process can be optimized using the Symmetric Delete algorithm used by SymSpell. GitHub

5. Grammar Checking

The system can be extended beyond spelling correction to identify grammar errors.

6. Sentence-Level Analysis

The program can analyze complete sentences and provide context-aware corrections.

7. Multiple Languages

Support for languages other than English can be added by using language-specific dictionaries.

8. PDF and DOCX Support

The system can be extended to process PDF and Microsoft Word documents instead of only .txt files.

9. User Dictionary

Users can be allowed to add custom words to the dictionary.

10. Advanced User Interface

The current console-based system can later be converted into a desktop or web application.

11. Machine Learning

A future version could combine edit distance with language models or machine-learning techniques to select corrections based on context.

12. Real-Time Spell Checking

The system could be integrated into a text editor so that spelling mistakes are detected while the user types.

9. REFERENCES

1. Levenshtein, V. I.

"Binary Codes Capable of Correcting Deletions, Insertions, and Reversals." Soviet Physics Doklady, 1966.

Levenshtein Edit Distance Reference

---

2. Peter Norvig

"How to Write a Spelling Corrector."

Peter Norvig – How to Write a Spelling Corrector

---

3. Hunspell

Open-source spell checking and morphological analysis library.

Hunspell GitHub Repository

---

4. SymSpell

Spelling Correction and Fuzzy Search using the Symmetric Delete algorithm.

SymSpell GitHub Repository

---

5. Burkhard and Keller

“Some Approaches to Best-Match File Searching.” Communications of the ACM, 1973.

Burkhard-Keller Tree Reference

---

6. LanguageTool

Online writing assistant for spelling, grammar, style, and writing suggestions.

LanguageTool

---

7. Wagner and Fischer

“The String-to-String Correction Problem.” Journal of the ACM, 1974.

DOI: 10.1145/321796.321811

---

8. Hamming, R. W.

“Error Detecting and Error Correcting Codes.” Bell System Technical Journal, 1950.

DOI: 10.1002/j.1538-7305.1950.tb00463.x